IMPERIAL

Ports and Adapters Architecture for Image Analysis

Volodymyr Nechyporuk-Zloy
Facility for Imaging by Light Microscopy
@vnzloy

Introduction: Ports and Adapters Architecture

FAIR Principles in Image Analysis

- Findability: Resources are described with rich metadata and persistent identifiers. This enables efficient discovery by researchers and software systems.
- Accessibility: Resources can be retrieved through standardized and documented access mechanisms. Access conditions and permissions are clearly defined.
- Interoperability: Resources use standardized formats, metadata, and interfaces that enable information exchange between software systems. This ensures technical compatibility across tools and platforms.
- Reusability: Resources contain sufficient metadata, provenance, and documentation to support future use. This enables reproducibility and maximizes long-term scientific value.

Critical Image Analysis

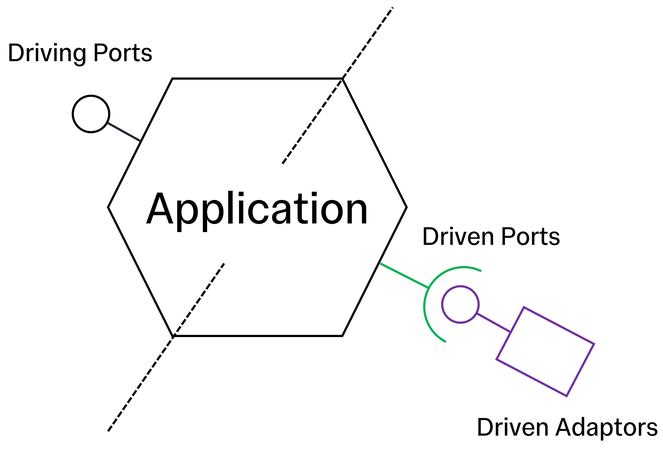
- Image analysis in domains where accuracy, reproducibility, and transparency are essential for decision-making. Errors may have significant scientific, clinical, or operational consequences.
- Examples:
  - Health care
  - Drug discovery

Software Architecture

- Separation of concerns: Organizes functionality into distinct components with clear responsibilities. This improves maintainability, readability, and scalability.
- Information flow: Defines how data and control signals move through the system. Well-defined flows improve reliability, observability, and system understanding.
- Dependency management: Controls relationships between components to reduce coupling and complexity. This facilitates system evolution, testing, and technology replacement over time.

Ports and Adapters Architecture

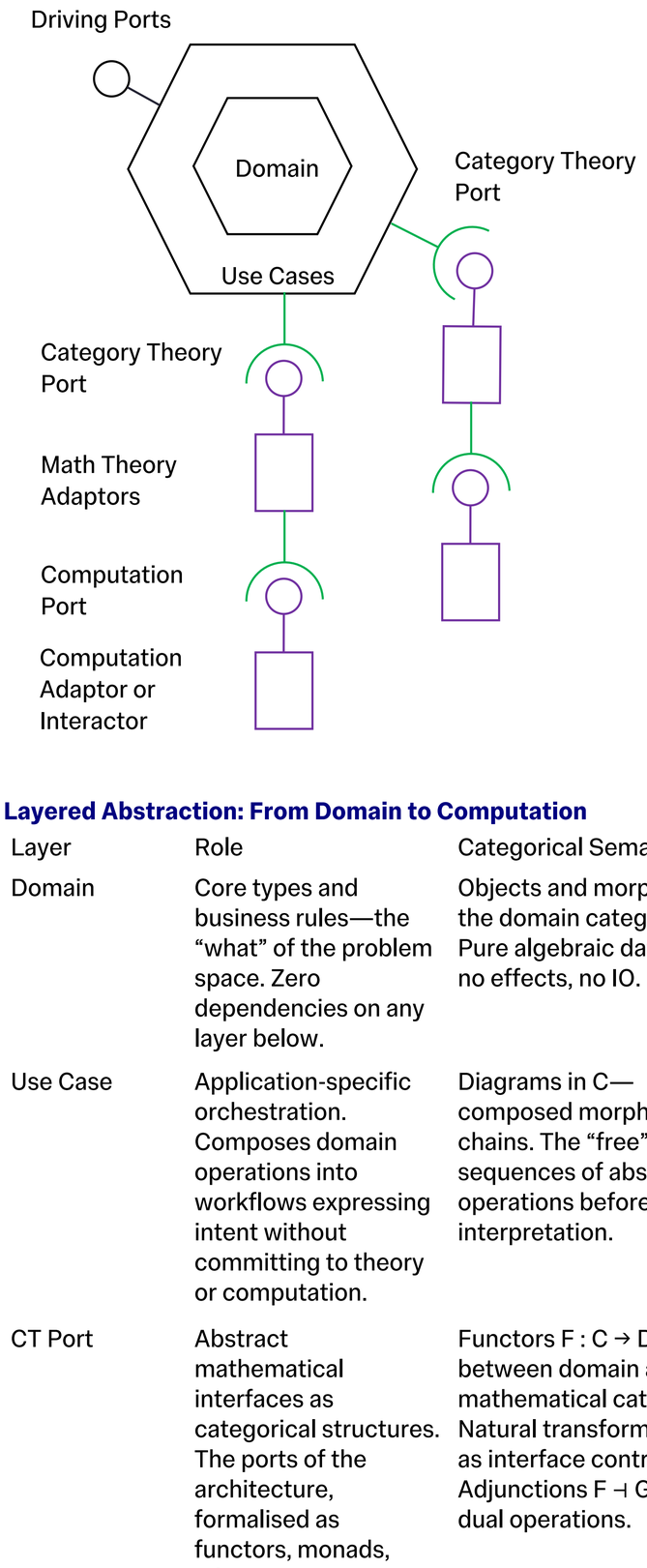
- Core idea: The application should not depend directly on external systems. Instead, it communicates through ports (interfaces), while adapters translate between the application and specific technologies. Port can also directly interact with piece of the software – interactor.
- Terms:
  - Application: The software containing all domain logic.
  - Primary (Driving) port: A port used to make request of the app.
  - Secondary (Driven) port: A port used to make request of the driven adapters (or interactors).
  - Primary (Driving) adapter: An adapter used to make request of the app.
  - Secondary (Driven) adapter: An adapter used to make request of the driven adapters (or interactors).
  - Dependency configurator: The unique component that instantiates all driven adapters, the application, and all driving adapters, injecting each into its dependants, such that no other component performs cross-boundary wiring.



Results: 6-Layer Architecture

6-Layer Architecture

- Core idea: The 6-layer architecture is a modified ports-and-adapters architecture that separates an image analysis pipeline into six independently replaceable layers — domain, use case, category theory (CT) port, mathematical theory (MT) adapter, computation (Comp) port, computation adapter — by making the boundaries between mathematical theories and between mathematical specification and hardware execution explicit by means of category theory rather than implicit.



Layered Abstraction: From Domain to Computation

Layer	Role	Categorical Semantics
Domain	Core types and business rules—the “what” of the problem space. Zero dependencies on any layer below.	Objects and morphisms of the domain category C. Pure algebraic data types; no effects, no IO.
Use Case	Application-specific orchestration. Composes domain operations into workflows expressing intent without committing to theory or computation.	Diagrams in C— composed morphism chains. The “free” layer: sequences of abstract operations before interpretation.
CT Port	Abstract mathematical interfaces as categorical structures. The ports of the architecture, formalised as functors, monads, adjunctions.	Functors F : C → D between domain and mathematical categories. Natural transformations as interface contracts. Adjunctions F ⊣ G pairing dual operations.
Math Adapter	Concrete mathematical theories that interpret the categorical ports. Each adapter selects a specific branch of mathematics.	Natural transformations η : F ⇒ G (interpreters). F-algebras (C, α : FC → C) carrying algebraic structure into the domain.
Comp Port	Abstract computational interfaces—array ops, convolution, optimisation— independent of any backend. Monoidal structure enables parallelism.	Functors from the math category to a computation category Comp. Monoidal structure (Comp, ⊗, I) for composing and parallelising.
Comp Adapter or Interactor	Concrete backends that execute computations. Swappable without touching any layer above.	Natural transformations from Comp to Set (or Hask). F-algebras μ : F(X) → X evaluating the free computation into concrete values.

Layer	Imaging Example	Haskell / Agda
Domain	`Image a`, `Kernel a`, `Mask`, `StructuringElement`, `PixelType`. Pure functions between domain types.	data Image a = Image (V.Vector a) data Kernel a = Kernel (Matrix a) -- Agda: record Image (A : Set)
Use Case	“Smooth, then segment, then measure” as abstract pipeline.	pipeline :: Sem '[Smooth, Segment] Result pipeline = smooth >>= segment >>= measure -- Free composition, no interpreter yet
CT Port	`class Lattice l`— complete lattice port shared by PDE smoothing and morphological opening. `Adjunction` between smoothing ⊣ sharpening.	class CompleteLattice l where sup :: Set l -> l inf :: Set l -> l -- Agda: record Functor (F : Set → Set)
Math Adapter	PDE theory (heat equation → Gaussian), lattice theory (complete lattice → morphological opening), optimal transport (Wasserstein → registration).	interpPDE :: Sem (PDEEffect ': r) a -> Sem r a interpLattice :: Sem (LatticeEffect ': r) a -> Sem r a
Comp Port	Abstract `convolve`, `fft`, `solve`, `optimize`. Backend-agnostic array operations.	data ArrayOp m a where Convolve :: Arr -> Arr -> ArrayOp m Arr FFT :: Arr -> ArrayOp m Arr -- Polysemy GADTs effect
Comp Adapter or Interactor	JAX (`jit`, `vmap`, `grad`), NumPy, CuPy, NIS-Elements PythonScript API, Leica LAS X macro engine.	runJAX :: Sem (ArrayOp ': r) a -> Sem r a runNumPy :: Sem (ArrayOp ': r) a -> Sem r a

Compositional Guarantees via Category Theory

- Dependency rule: Each layer is a functor whose domain is the layer above. Information flows downward through natural transformations. The Fubini theorem for coends guarantees interpreter composition commutes—adapters at layers 4 and 6 are independently swappable.
- Cross-field composition: PDE-based Gaussian smoothing ∘ lattice-theoretic morphological opening composes at Layer 3 via the shared CompleteLattice port, even though the two Math Adapters at Layer 4 draw on entirely different branches of mathematics.

Conclusion

- The architecture is applicable to image analysis generally, but is especially relevant for critical applications — healthcare, drug discovery — where errors carry significant scientific or clinical consequences and rigorous guarantees on reproducibility and correctness are required.
- Principled swappability. The Fubini theorem for coends guarantees that Math Theory Adapters (layer 4) and Computation Adapters (layer 6) compose independently — theories and backends are swapped without altering any other layer.
- Cross-field composition. Operations from different mathematical branches (e.g. PDE theory and lattice theory) compose at the CT Port layer whenever they share a common categorical structure, enabling rigorously typed multi-theory pipelines.
- FAIR alignment. Ports supply standardised interfaces (Interoperability, Accessibility); categorical semantics provide unambiguous per-step metadata (Findability); independently replaceable adapters with full provenance support Reusability.